

AI Voicebot Development Screening Project

LiveKit Voice Agent - Multilingual Interactive Platform

Executive Summary

This screening project challenges interns to develop a sophisticated multilingual voicebot using LiveKit as the core real-time communication framework. The bot must demonstrate advanced conversational capabilities across English and Indian languages (Hindi), with optional support for a third language of choice. The project evaluates technical competency, attention to scope management, language processing expertise, and ability to deliver production-grade voice AI solutions.

1. Project Overview

1.1 Vision

Create a fully functional, professional-grade AI voicebot capable of conducting natural, respectful conversations in multiple languages while maintaining strict adherence to defined business scenarios. The bot should replicate human conversational patterns with Indian accent authenticity, seamless language switching, and contextual awareness.

1.2 Business Scenarios

Interns will select **ONE** of the following scenarios to build their voicebot solution:

Option A: Presale Bot

- Engage prospective customers with product information
- Identify customer needs and pain points
- Qualify leads through conversational discovery
- Schedule follow-up consultations
- **Scope Boundary:** Conversational discovery only; no pricing negotiations or sales closure

Option B: Sales Bot (Product Pitch)

- Deliver compelling product demonstrations via voice
- Address common customer objections
- Highlight key product benefits and features
- Guide customers through product capabilities
- **Scope Boundary:** Educational product information; no contract negotiation or order processing

Option C: Marketing Bot (Lead Nurturing)

- Engage existing leads with personalized messaging
- Share relevant case studies, use cases, and success stories
- Provide educational content about industry trends
- Determine interest level for sales team handoff
- **Scope Boundary:** Educational content and relationship building; no direct sales pitch

1.3 Mandatory Features

All voicebots, regardless of scenario selection, **MUST** include:

1. **Bilingual Capability**

- Primary languages: English and Hindi
- Seamless language switching during conversation
- Automatic language detection (with user confirmation option)
- Context preservation across language switches

2. Indian Accent Integration

- Text-to-speech (TTS) engine configured for Indian English accent
- Natural prosody and intonation reflecting Indian English patterns
- Culturally appropriate vocabulary and phrasing

3. Professional Tone and Respect

- Never exhibit rudeness, sarcasm, or dismissive language
- Maintain polite, respectful responses to all user inputs
- Gracefully handle difficult or off-topic queries without negativity
- Include apology and redirection phrases when out of scope

4. Strict Scope Adherence

- No information provided outside selected scenario scope
- Clear, professional redirection when queries fall outside boundaries
- Predefined list of out-of-scope topics
- Escalation protocol for complex or sensitive questions

2. Project Scope

2.1 Core Deliverables

Tier 1: Essential Components (Must Have)

Component	Description	Status
LiveKit Integration	Complete setup of LiveKit Agents framework with real-time voice pipeline	Required
STT (Speech-to-Text)	Multi-language speech recognition (English, Hindi)	Required
LLM Integration	Large Language Model for context-aware responses	Required
TTS (Text-to-Speech)	Indian-accented voice output with language-specific voices	Required
Language Switching	Seamless detection and switching between English and Hindi	Required
Scenario-Specific Prompt	Detailed system prompt for selected business scenario	Required
Scope Boundary Enforcement	Mechanism to prevent out-of-scope information sharing	Required
Error Handling	Graceful handling of speech recognition errors, no-speech detection, background noise	Required
Conversation Logging	Complete audit trail of conversations with timestamps	Required
Basic Analytics	Call duration, language switches, topic categories tracked	Required

Tier 2: Enhanced Features (Nice to Have)

- Sentiment analysis of user input
- Conversation summarization and key point extraction
- Third language support (Tamil, Telugu, Bengali, Marathi, Kannada, or choice of intern)

- Advanced tone adaptation based on user sentiment
- Dynamic context management across multi-turn conversations
- Custom vocabulary and pronunciation guides
- Voice fingerprinting for repeat caller identification
- Real-time transcription display
- Conversation quality scoring

2.2 Out-of-Scope Items

The following are explicitly **NOT** part of this project:

- Full-stack web application with user dashboards (basic CLI interface acceptable)
- SMS/Email integration (voice-only channel)
- Payment processing or transaction handling
- Human handoff to live agents (optional as future enhancement)
- Mobile application development
- Database optimization for massive scale (>1M concurrent users)
- Advanced machine learning model training from scratch
- Multi-channel bot synchronization (focus on voice channel only)

2.3 Scenario-Specific Scope Details

Presale Bot Scope

What the bot CAN do:

- Ask qualifying questions about company size, industry, current solutions
- Describe product use cases and general capabilities
- Share educational resources and documentation links
- Schedule meetings with sales team
- Provide contact information for sales representative

What the bot CANNOT do:

- Quote pricing or discuss discount structures
- Close sales or process orders
- Provide competitive analysis or negative competitor commentary
- Commit to specific implementation timelines
- Negotiate contract terms

Sales Bot Scope

What the bot CAN do:

- Present product features and benefits in detail
- Answer technical questions about product functionality
- Share case studies and success metrics
- Address common objections with prepared responses
- Demonstrate product workflows via description
- Provide technical documentation and setup guides

What the bot CANNOT do:

- Process orders or invoices
- Discuss pricing and payment terms
- Negotiate contract modifications
- Provide legal or compliance advice
- Make claims beyond verified product capabilities
- Offer custom development commitments

Marketing Bot Scope

What the bot CAN do:

- Share success stories and ROI metrics
- Provide industry insights and trend information
- Offer educational webinars or training resources
- Identify lead pain points and interests
- Gauge readiness for sales conversation
- Provide general product overview

What the bot CANNOT do:

- Provide specific pricing information
- Offer personalized product recommendations without additional context
- Make binding commitments
- Discuss confidential client information
- Override company policies or terms

3. Technical Requirements

3.1 Technology Stack (Recommended)

Layer	Technology	Alternative Options
Voice Framework	LiveKit Agents	-
Speech-to-Text	Deepgram, Google Cloud Speech-to-Text, Sarvam AI	AssemblyAI, AWS Transcribe
Large Language Model	OpenAI GPT-4o, Claude 3 Opus, Gemini 1.5	Llama 2, Mistral, Local LLM
Text-to-Speech	ElevenLabs (Indian voices), Sarvam AI (Indian languages), Google Cloud TTS	Azure Speech, AWS Polly, Voice.ai
Language Detection	Native language detection from STT confidence scores	Langdetect library, TextBlob
Backend Framework	Python with FastAPI or Flask	Node.js with Express, Go
Runtime Environment	Python 3.10+	-
Deployment	Docker containerization, AWS/GCP or local server	Kubernetes, Modal, Replit
Version Control	Git & GitHub	GitLab, Bitbucket

3.2 Environment Setup

Required Accounts & Credentials:

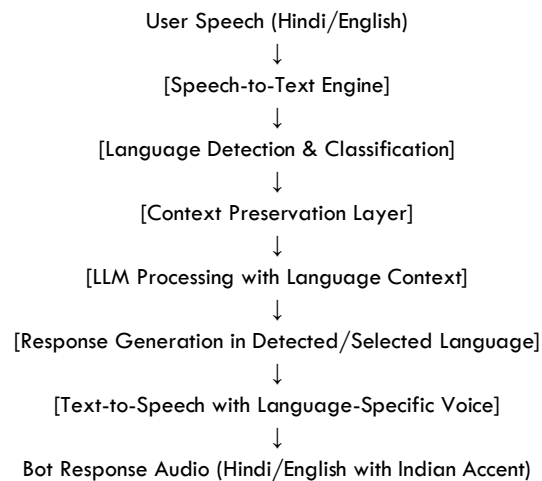
- LiveKit Cloud account or self-hosted instance

- Speech-to-Text API credentials (Deepgram, Google Cloud, or equivalent)
- LLM API access (OpenAI, Anthropic, Google, etc.)
- Text-to-Speech API credentials (ElevenLabs, Sarvam AI, or equivalent)

Local Development Requirements:

- Python 3.10 or higher
- Virtual environment (venv or Poetry)
- Git for version control
- Terminal/CLI familiarity
- Basic understanding of WebRTC and RTC protocols

3.3 Multilingual Processing Architecture



3.4 Core Components to Implement

1. LiveKit Voice Agent Setup

- Initialize LiveKit room and participant management
- Configure WebRTC connection for low-latency audio
- Implement voice activity detection (VAD)
- Set up audio streaming pipeline (inbound/outbound)

2. Speech-to-Text Pipeline

- Stream audio from LiveKit to STT service
- Handle real-time transcription
- Implement language detection
- Manage partial and final transcripts
- Error handling for poor audio quality

3. Language Detection & Switching

- Detect language from STT confidence scores
- Maintain conversation language preference
- Allow mid-conversation language switches
- Preserve context when switching languages
- Store language preference per session

4. LLM Processing Engine

- System prompt tailored to scenario and scope
- Context management for multi-turn conversations
- Temperature and token limits for response control
- Scope validation layer (check if response is in-scope)
- Response safety filters

5. Text-to-Speech Output

- Generate speech in user's language
- Apply Indian English accent parameters
- Adjust speech rate and intonation
- Stream audio back through LiveKit

6. Conversation State Management

- Track conversation history and context
- Maintain user intent and extracted entities
- Store language preference and switches
- Log conversation metadata (duration, language switches, sentiment)

7. Scope Validation Layer

- Define allowed topics and information domains
- Implement content filtering for each scenario
- Generate out-of-scope redirect responses
- Log scope violations for analysis

3.5 Data Structures & APIs

Conversation Context Object:

```
{
  "conversation_id": "UUID",
  "current_language": "en" | "hi",
  "language_history": ["en", "hi", "en"],
  "user_intent": "string",
  "entities_extracted": {key: value},
  "conversation_history": [
    {"role": "user", "content": "...", "language": "en"},
    {"role": "assistant", "content": "...", "language": "en"}
  ],
  "scenario": "presale" | "sales" | "marketing",
  "conversation_start_time": "timestamp",
  "turn_count": "integer",
  "sentiment": "positive" | "neutral" | "negative"
}
```

4. Performance Parameters

4.1 Latency Requirements

Metric	Target	Acceptable Range	Notes
End-to-End Latency	< 1.5 seconds	< 2.5 seconds	Time from user stops speaking to bot starts responding

Speech-to-Text Latency	< 500ms	< 800ms	Time to convert speech to text
LLM Response Time	< 800ms	< 1.2 seconds	Time to generate LLM response
Text-to-Speech Latency	< 300ms	< 500ms	Time to synthesize and stream audio
Language Detection Time	< 100ms	< 200ms	Included in STT latency

4.2 Accuracy Metrics

Metric	Target	Measurement
Speech Recognition Accuracy	> 90%	English: >92%, Hindi: >85%
Language Detection Accuracy	> 98%	Correctly identifies language in bilingual context
Scope Compliance	100%	All responses within defined scope; no out-of-scope information leaked
Tone Appropriateness	> 95%	Manual review: responses rated as respectful and professional
Language Switch Success	100%	All requested language switches execute without errors
Context Preservation	> 90%	Bot maintains conversation context after language switches

4.3 Reliability & Stability

Metric	Target	Acceptable
Uptime	99%	95% minimum
Error Recovery	Graceful	No crashes; attempt retry before escalation
Maximum Call Duration	30+ minutes	No time limits or disconnections
Concurrent Calls (Testing)	5+	3+ minimum
Memory Stability	No memory leaks	Memory usage stable over extended calls

4.4 User Experience Metrics

Metric	Target	Measurement
Natural Speech Pacing	Natural rhythm	~150-160 words per minute for Indian English
Pronunciation Accuracy	> 95%	Indian English phonetics correctly applied
Response Relevance	> 90%	Human evaluation of answer appropriateness
Turn-taking Smoothness	Smooth	No awkward pauses or overlapping speech
Language Switching Smoothness	Seamless	No delays or repetition during switches

5. Success Parameters & Evaluation Criteria

5.1 Technical Success Criteria (50% Weight)

Functionality (20 points)

- ☐ LiveKit integration complete and stable (5 points)
- ☐ Real-time speech-to-text working in English and Hindi (5 points)
- ☐ LLM integration producing contextual responses (5 points)
- ☐ Text-to-speech with Indian accent operational (5 points)

Multilingual Features (15 points)

- ☐ Language detection working with >95% accuracy (5 points)
- ☐ Seamless language switching within conversation (5 points)
- ☐ Context preservation across language switches (5 points)

Scope Management (15 points)

- ☐ Scenario-specific system prompt well-defined (5 points)
- ☐ Out-of-scope queries handled gracefully (5 points)
- ☐ No information leakage outside scope (5 points)

Code Quality (10 points)

- ☐ Clean, documented, production-ready code (5 points)
- ☐ Proper error handling and logging (5 points)

5.2 Performance Success Criteria (20% Weight)

Latency (10 points)

- ☐ End-to-end latency < 2.5 seconds consistently (10 points)
- Deduction if latency exceeds thresholds (scale: -2 points per 0.5 second over limit)

Accuracy (10 points)

- ☐ Speech recognition accuracy > 85% (5 points)
- ☐ Language detection > 95% accurate (5 points)

5.3 Business Requirements Success (20% Weight)

Tone & Professionalism (10 points)

- ☐ All responses maintain respectful, professional tone (10 points)
- ☐ Manual review: No rude, dismissive, or sarcastic responses (-5 points per instance)
- ☐ Appropriate handling of difficult questions (5 points)

Indian Accent Implementation (10 points)

- ☐ Clear Indian English accent in TTS output (7 points)
- ☐ Natural Indian English phrasing and vocabulary (3 points)

5.4 Documentation & Presentation (10% Weight)

Technical Documentation (5 points)

- ☐ Architecture diagram and component descriptions (2 points)
- ☐ Setup and deployment instructions (1.5 points)
- ☐ API and configuration documentation (1.5 points)

Project Presentation (5 points)

- ☐ Live demonstration of all features (2 points)
- ☐ Explanation of design decisions (1.5 points)
- ☐ Future improvement roadmap (1.5 points)

5.5 Bonus Points (Up to 10 points)

- ☐ Third language support (Tamil, Telugu, Bengali, etc.) (+5 points)
- ☐ Sentiment analysis of user inputs (+3 points)

- ☐ Conversation summarization feature (+3 points)
- ☐ Advanced tone adaptation (+2 points)
- ☐ Production-grade deployment (AWS/GCP with monitoring) (+4 points)
- ☐ Comprehensive unit and integration tests (+3 points)
- ☐ Video demo with explanation (+2 points)

5.6 Evaluation Matrix

Category	Excellent (90-100)	Good (75-89)	Acceptable (60-74)	Below Target (<60)
Functionality	All features work perfectly; edge cases handled	Core features work; minor bugs fixed	Core features work; some edge cases unhandled	Critical features missing or broken
Performance	All latency/accuracy targets met	Mostly targets met; minor gaps	Some targets missed; acceptable performance	Significant performance issues
Code Quality	Production-ready; well-documented; scalable	Good structure; mostly documented; minor cleanup needed	Functional but messy; limited documentation	Poor structure; undocumented; difficult to maintain
Scope Adherence	Perfect scope enforcement; zero violations	Excellent scope adherence; very rare violations	Good adherence; occasional out-of-scope responses	Frequent scope violations or information leakage
Tone & Respect	Always professional; excellent handling of edge cases	Professional throughout; handles most edge cases well	Mostly professional; occasional tone issues	Frequent tone problems or rudeness
Documentation	Comprehensive; clear; deployment-ready	Good documentation; clear instructions	Basic documentation; mostly clear	Minimal or unclear documentation

5.7 Pass/Fail Criteria

Project is considered **SUCCESSFUL** if:

- ☒ All Tier 1 (Essential) components are implemented and working
- ☒ Technical functionality score $\geq 50/50$ points
- ☒ Scope compliance is 100% (zero out-of-scope responses in testing)
- ☒ Tone and professionalism maintained throughout (no rude responses)
- ☒ Indian accent clearly present in TTS output
- ☒ Bilingual capabilities (English & Hindi) functioning
- ☒ End-to-end latency < 3 seconds on 80% of interactions
- ☒ Total score $\geq 70/100$ points

Project requires revision if:

- ☒ Critical functionality is missing or broken
- ☒ Out-of-scope information is being provided
- ☒ Tone/respect requirements violated
- ☒ Bilingual capabilities not functioning
- ☒ Total score < 70/100 points

6. Deliverables Checklist

6.1 Code & Implementation

- ☐ Complete, working codebase in GitHub repository
- ☐ .env.example file with required credentials structure
- ☐ requirements.txt or pyproject.toml with dependencies
- ☐ Dockerfile for containerized deployment
- ☐ Configuration files for different scenarios (presale/sales/marketing)
- ☐ Conversation logging and persistence mechanism
- ☐ README with setup and usage instructions

6.2 Configuration & Prompts

- ☐ Scenario-specific system prompt (saved as JSON or YAML)
- ☐ Out-of-scope topic definitions and redirect responses
- ☐ Tone and behavior guidelines embedded in prompt
- ☐ Conversation flow examples (test conversations)
- ☐ Indian accent and language-specific configuration

6.3 Testing & Quality Assurance

- ☐ Test conversation transcripts (minimum 5 per scenario)
- ☐ Performance metrics report (latency, accuracy, uptime)
- ☐ Scope compliance test results
- ☐ Language switching test cases
- ☐ Error scenario handling documentation

6.4 Documentation

- ☐ System architecture diagram (Lucidchart, draw.io, or similar)
- ☐ Component interaction diagrams
- ☐ Deployment guide (local and cloud options)
- ☐ API reference (if applicable)
- ☐ Troubleshooting guide
- ☐ Future improvement suggestions

6.5 Presentation Materials

- ☐ Live demo credentials and setup
- ☐ Presentation slides (5-10 slides max)
- ☐ Video walkthrough (3-5 minutes optional)
- ☐ Q&A preparation document

7. Submission Requirements

7.1 Repository Structure

```
voicebot-screening-project/
├── src/
│   ├── main.py
│   ├── livekit_manager.py
│   └── stt_engine.py
```

```
|  ├── llm_processor.py
|  ├── tts_engine.py
|  ├── language_detector.py
|  ├── conversation_manager.py
|  ├── scope_validator.py
|  ├── config/
|  |   ├── scenarios/
|  |   |   ├── presale_config.yaml
|  |   |   ├── sales_config.yaml
|  |   |   └── marketing_config.yaml
|  |   ├── prompts/
|  |   |   ├── presale_system_prompt.txt
|  |   |   ├── sales_system_prompt.txt
|  |   |   └── marketing_system_prompt.txt
|  |   └── settings.py
|  ├── tests/
|  |   ├── test_stt.py
|  |   ├── test_llm.py
|  |   ├── test_language_switching.py
|  |   └── test_scope_validation.py
|  ├── logs/
|  |   └── .gitkeep
|  ├── docs/
|  |   ├── ARCHITECTURE.md
|  |   ├── SETUP.md
|  |   ├── DEPLOYMENT.md
|  |   └── TROUBLESHOOTING.md
|  ├── .env.example
|  ├── Dockerfile
|  ├── requirements.txt
|  └── README.md
```

7.2 Documentation Format

- **README.md:** Markdown format, GitHub-rendered
- **Architecture:** Diagrams in PNG/SVG format or embedded Mermaid diagrams
- **Deployment Guide:** Step-by-step, numbered instructions
- **Prompts:** Human-readable text or YAML format with clear structure

7.3 Code Standards

- **Language:** Python 3.10+
- **Style:** PEP 8 compliant
- **Comments:** Docstrings for all functions; inline comments for complex logic
- **Error Handling:** Try-catch blocks with meaningful error messages
- **Logging:** Structured logging with timestamp, level, and message

8. Timeline & Milestones

Week	Milestone	Deliverables
Week 1	Environment Setup & Architecture Design	Development environment ready; Architecture diagram; Tech stack approved
Week 2	LiveKit & Speech-to-Text Integration	Voice pipeline functional; Real-time transcription working
Week 3	LLM Integration & Language Processing	LLM responding; Language detection working; Basic scope validation

Week 4	Multilingual Features & TTS Integration	Language switching tested; Indian accent TTS output; Conversation logging
Week 5	Testing & Optimization	Performance optimizations; Latency < 2.5s; Accuracy > 90%; Test suite complete
Week 6	Documentation & Final Presentation	All documentation complete; Final code cleanup; Presentation slides ready

9. Resources & References

9.1 Official Documentation

- **LiveKit Agents:** <https://docs.livekit.io/agents/>
- **Deepgram STT:** <https://developers.deepgram.com/>
- **ElevenLabs TTS:** <https://elevenlabs.io/docs>
- **OpenAI GPT API:** <https://platform.openai.com/docs/>
- **Sarvam AI (Indian Languages):** <https://docs.sarvam.ai/>

9.2 Recommended Libraries

LiveKit

livekit
livekit-agents
livekit-agents-deepgram
livekit-agents-elevenlabs
livekit-agents-openai

LLM & Language Processing

openai
langdetect
python-dotenv

Backend & Utilities

fastapi
uvicorn
pydantic
pyyaml
python-logging-loki # for centralized logging

Testing

pytest
pytest-asyncio

9.3 Useful Tutorials & Articles

- "Building Human-Like AI Voice Agents: A Guide to Using LiveKit with GPT-4o" (2026)
- "Multilingual VoiceBot Best Practices: 16 Strategies to Maximize FCR" (2025)
- "Build and Deploy Real-Time AI Voice Agents Using LiveKit and AssemblyAI" (2025)
- "Building Multilingual Voicebots for Global Users" (2025)

9.4 Community & Support

- LiveKit Community Slack: <https://livekit.io/slack>
 - GitHub Issues & Discussions
 - Stack Overflow tags: `livekit`, `voice-ai`, `speech-to-text`
-

10. Assessment & Feedback

10.1 Evaluation Schedule

- **Mid-project Review (Week 3):** Architecture, progress, technical approach
- **Code Review (Week 5):** Code quality, structure, test coverage
- **Final Evaluation (Week 6):** Functionality, performance, presentation

10.2 Feedback Mechanism

- Weekly sync meetings (30 min) for guidance and troubleshooting
- Code review with detailed comments on GitHub
- Performance metrics analysis with recommendations
- Mentorship on production deployment practices

10.3 Success Tracking

- **Green:** On track, all deliverables met
 - **Yellow:** Slight delays, minor issues to address
 - **Red:** Significant challenges, intervention needed
-

11. Appendix: Sample Test Scenarios

Test Scenario 1: Presale Bot - English Conversation

User: "Hi, I'm looking for a solution to automate my customer calls"

Bot: "Hello! Thanks for reaching out. I'd be happy to help. Could you tell me a bit about your business and the types of calls you're looking to automate?"

User: "We get about 500 calls a day for appointment scheduling"

Bot: "That's great. A 500 daily call volume for scheduling is a perfect use case for automation. Our solution can handle appointment booking, rescheduling, and cancellations. Are you currently using a specific calendar system?"

User: "We use Google Calendar"

Bot: "Excellent. Our platform integrates seamlessly with Google Calendar. I'd love to schedule a brief demo with our sales team to show you how this works. What days next week work best for you?"

Test Scenario 2: Presale Bot - Hindi to English Switch

User: "Namaste, main aapke product ke baare mein jaanna chahta hoon" (Hi, I want to know about your product)

Bot: "Namaste! Mujhe khushi hai ki aap hamare baare mein jaanna chahte hain. Aap English mein baat karna pasand karenge ya Hindi mein? (I'm happy to help. Would you prefer to talk in English or Hindi?)"

User: "English please"

Bot: "Perfect! I'm happy to switch to English. So, what specific challenges are you facing with your current customer communication system?"

Test Scenario 3: Out-of-Scope Handling

User: "How much does your solution cost?"

Bot: "That's a great question about pricing. Pricing depends on your call volume and specific needs. I'm not able to quote pricing directly, but I can connect you with our sales team who can provide you with a customized proposal. Would you like me to schedule that conversation?"

User: "Can I get a 50% discount?"

Bot: "I appreciate the question! Pricing and discount discussions are best handled directly with our sales team. They can work with you to find a solution that fits your budget. Shall I set up a meeting for you?"
